

運用 TensorFlow Lite for Microcontrollers 和 SparkFun Edge 進行 AI 語音辨識

來源：<https://codelabs.developers.google.com/codelabs/sparkfun-tensorflow?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將學習如何在 SparkFun Edge 這個以電池供電的開發板上，使用 TensorFlow Lite for Microcontrollers 執行語音辨識模型。SparkFun Edge 是一個以電池供電的開發板，內含微控制器。

目錄

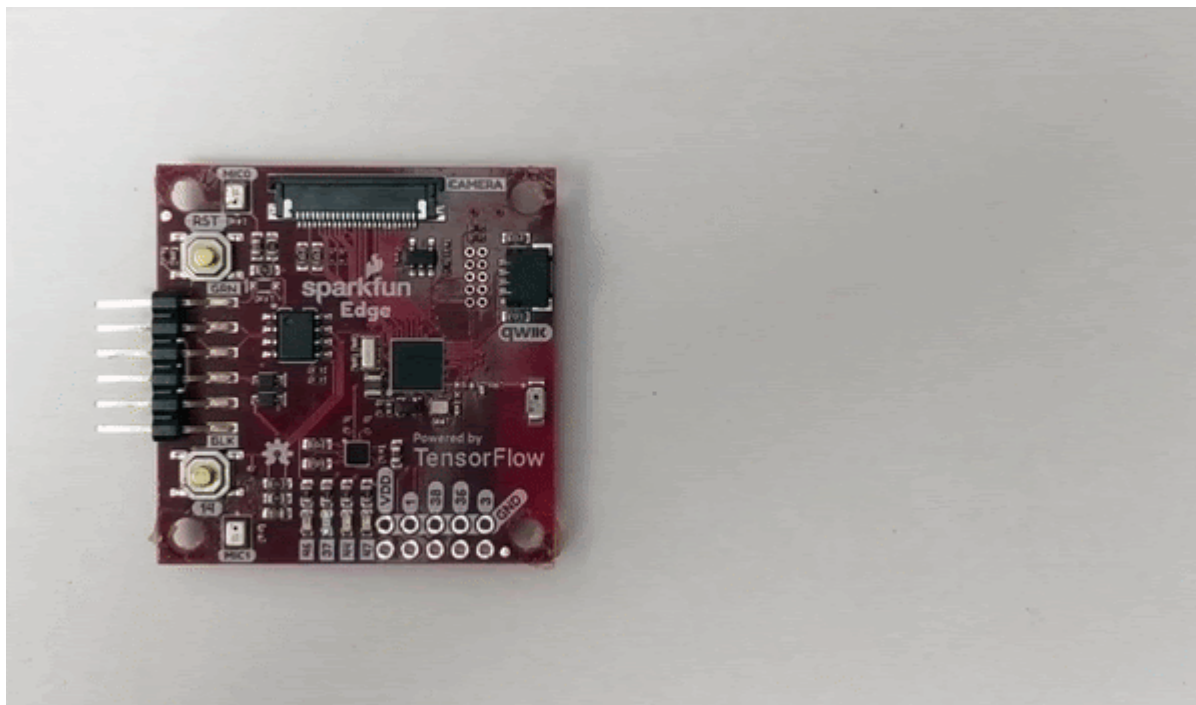
1. [1. 簡介](#)
2. [2. 設定硬體](#)
3. [3. 設定軟體](#)
4. [4. 建構及準備二進位檔](#)
5. [5. 前置準備刷入二進位檔](#)
6. [6. 刷入二進位檔](#)
7. [7. 示範](#)
8. [8. 讀取偵錯輸出內容 \(選用\)](#)
9. [9. 延長代碼效期 \(選用\)](#)
10. [10. 後續步驟](#)

步驟 1 · 約 10 分鐘

1. 簡介

建構項目

在本程式碼研究室中，我們將學習如何使用 [TensorFlow Lite For Microcontrollers](#)，在 [SparkFun Edge 開發板](#) 上執行深度學習模型。我們將使用開發板內建的語音偵測模型，透過卷積類神經網路偵測透過開發板兩個麥克風說出的「yes」和「no」字詞。



微控制器上的機器學習

機器學習技術可用於打造智慧型工具，讓使用者生活更輕鬆，例如 [Google 助理](#)。但通常這類體驗需要大量運算或資源，包括強大的雲端伺服器或桌機。不過，現在可以在微控制器等耗電量低的微型硬體上執行機器學習推論。

微控制器非常常見、價格低廉、耗電量極低，而且非常可靠。這些晶片內建於各種家用裝置，例如家電、汽車和玩具。事實上，每年生產的微控制器裝置約有 300 億部。



我們將機器學習技術導入微型微控制器，讓數十億部裝置變得更智慧，且不需使用昂貴的硬體或穩定的網際網路連線。想像一下：可依照您的日常作息進行調整的智慧應用程式、可區分問題與一般操作的智慧型工業用感應器，以及可讓兒童透過開心又有趣的方式學習的神奇玩具。

TensorFlow Lite For Microcontrollers (軟體)

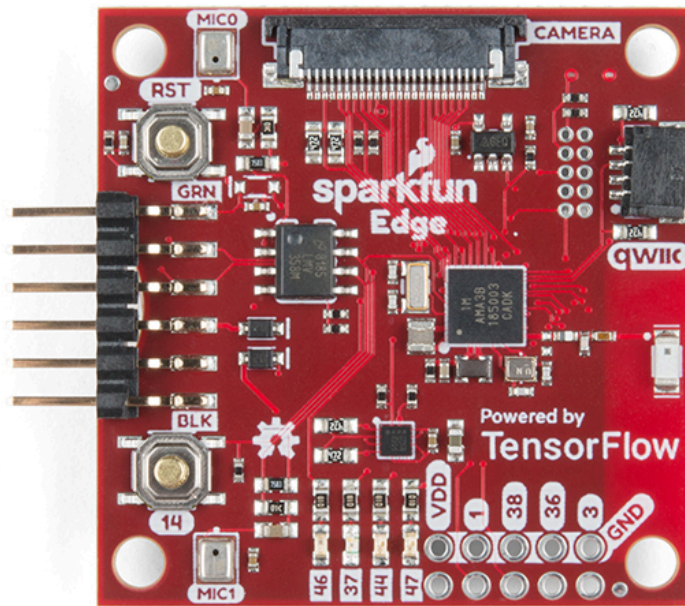


[TensorFlow](#) 是 Google 的開放原始碼機器學習架構，可用於訓練及執行模型。[TensorFlow Lite](#) 是一種軟體框架，是 TensorFlow 的最佳化版本，適用於在小型、相對低功耗的裝置 (例如手機) 上執行 TensorFlow 模型。

[TensorFlow Lite For Microcontrollers](#) 是一種軟體框架，是 TensorFlow 的最佳化版本，專門用於在微控制器等小型低功耗硬體上執行 TensorFlow 模型。它符合這些嵌入式環境的必要限制，也就是二進位檔大小較小，不需要作業系統支援、任何標準 C 或 C++ 程式庫，也不需要動態記憶體配置等。

SparkFun Edge (硬體)

[SparkFun Edge](#) 是以微控制器為基礎的平台，也就是單一電路板上的微型電腦。這類裝置配備處理器、記憶體和 I/O 硬體，可與其他裝置傳送及接收數位信號。這款充電器有四個可透過軟體控制的 LED 燈，並採用你喜愛的 Google 顏色。



與電腦不同，微控制器不會執行作業系統。您編寫的程式會直接在硬體上執行，您可以在電腦上編寫程式碼，然後透過名為「程式設計器」的裝置下載至微控制器。

微控制器並非功能強大的電腦，這類裝置的處理器很小，記憶體也不多。但由於微控制器的設計盡可能簡單，因此耗電量極低。視程式用途而定，SparkFun Edge 只要使用單一鈕扣型電池，就能運作數週！

課程內容

- 在電腦上編譯 SparkFun Edge 的範例程式
- 將程式部署至裝置
- 變更程式並重新部署

軟硬體需求

您需要下列硬體：

- Linux 或 MacOS 電腦
- [SparkFun Edge 開發板](#)
- [SparkFun USB-C Serial Basic](#) 程式設計器
- [USB-C 對 USB-A](#) 傳輸線 (如果使用 USB-C 電腦，請改用 [USB-C 對 USB-C](#) 傳輸線)
- (選用) [3V 20mm 鈕扣型鋰電池 \(CR2032\)](#)，可在沒有程式設計師和傳輸線的情況下執行推論

您需要下列軟體：

- Git (在指令列中執行 `git`，檢查是否已安裝)
- Python 3 (在指令列中執行 `python3` 或 `python --version`，檢查是否已安裝)
- Python 3 適用的 Pip ([實用的 StackOverflow 回答](#))
- 4.2.1 以上版本 (在指令列執行 `make --version`，檢查是否已安裝)
- [SparkFun Serial Basic 驅動程式](#)

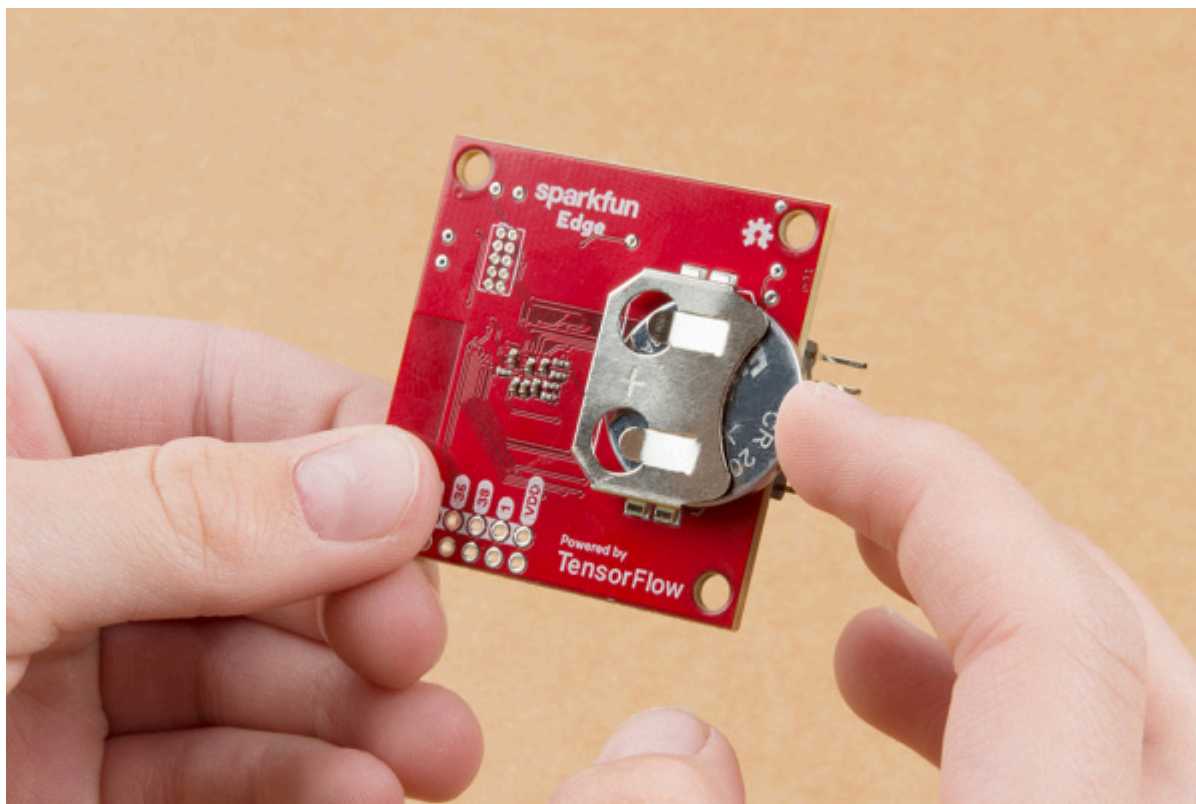
步驟 2 · 約 10 分鐘

2. 設定硬體

SparkFun Edge 微控制器預先安裝了可執行語音模型的二進位檔。在覆寫這個模型之前，請先執行這個模型。

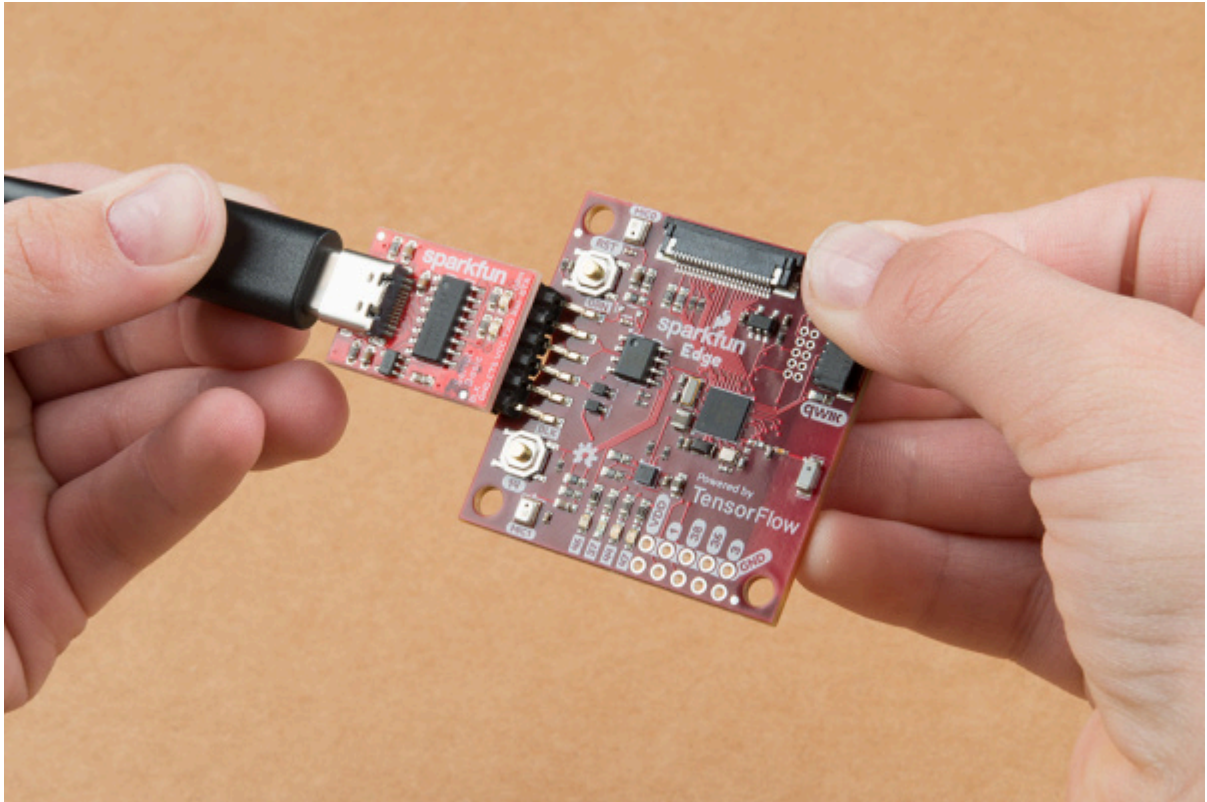
為開發板供電：

1. 將鈕扣型電池插入電路板背面的電池接頭 (電池的「+」面必須朝上)。如果主機板已裝有電池，請取出塑膠片，並將電池完全插入)



注意：SparkFun Edge 開發板沒有關機開關，因此不使用時請取出電池，以節省電力。如要取出電池，請使用筆、鉛筆或鑷子等小型工具，將電池從接頭推出。請小心不要損壞電路板，或讓電池兩側短路。

2. 如果沒有鈕扣電池，可以使用 SparkFun USB-C Serial Basic 程式設計師裝置為電路板供電。如要將裝置連接至開發板，請按照下列步驟操作：
 - 找出 SparkFun Edge 側邊的六針接頭。
 - 將 SparkFun USB-C Serial Basic 插入這些接腳，確保每個裝置上標示為「BLK」和「GRN」的接腳正確對齊。
 - 使用 USB-C 傳輸線連接 SparkFun USB-C Serial Basic 和電腦。



插入電池或連接 USB 程式設計器，為開發板供電後，開發板就會喚醒並開始透過麥克風監聽。藍燈應會開始閃爍。

開發板上的機器學習模型經過訓練，可辨識「是」和「否」這兩個字，並偵測語音。並透過亮起彩色 LED 燈來傳達結果。下表說明各個 LED 燈色的意義：

偵測結果	LED 燈顏色
「是」	黃色
「否」	紅色
不明語音	綠色
偵測不到語音	LED 燈未亮起

歡迎體驗

將板子舉到嘴巴前，然後說幾次「是」。黃色 LED 燈會閃爍。如果說「是」後沒有任何反應，請嘗試下列做法：

- 將板子放在距離嘴巴約 10 吋的位置
 - 避免背景噪音過多
 - 快速連續說出「是」幾次 (嘗試說「是是是」)
-

3. 設定軟體

我們現在要自行在微控制器上，下載、安裝及執行語音模型。為此，我們首先要下載這個程式的原始碼，以及建構程式所需的依附元件。該程式是以 C++ 編寫，必須先編譯成二進位檔，才能下載到開發板。二進位檔是含有程式的檔案，可由 SparkFun Edge 硬體直接執行。

以下操作說明適用於 Linux 或 macOS。

下載 TensorFlow 存放區

您可以在 GitHub 的 TensorFlow 存放區中找到程式碼，位置如下：

<https://github.com/tensorflow/tensorflow/tree/master/tensorflow/lite/micro>

在電腦上開啟終端機，切換至您通常儲存程式碼專案的目錄，下載 TensorFlow 存放區並輸入建立的目錄，如下所示：

```
cd ~ # change into your home (or any other) directory
git clone --depth 1 https://github.com/tensorflow/tensorflow.git
cd tensorflow
```

下載 Python 依附元件

我們將使用 Python 3 準備二進位檔，並將其刷入裝置。Python 指令碼必須有特定程式庫才能運作。執行下列指令安裝這些依附元件：

```
pip3 install pycrypto pyserial --user
```

步驟 4 · 約 10 分鐘

4. 建構及準備二進位檔

我們將建構二進位檔，並執行指令，準備將二進位檔下載至裝置。

建構二進位檔

如要下載所有必要依附元件並建立二進位檔，請執行下列指令：

```
make -f tensorflow/lite/micro/tools/make/Makefile TARGET=sparkfun_edge  
micro_speech_bin
```

如果建構作業順利完成，輸出內容的最後一行應如下所示：

```
arm-none-eabi-objcopy tensorflow/lite/micro/tools/make/gen/sparkfun_edge_cortex-  
m4/bin/micro_speech tensorflow/lite/micro/tools/make/gen/sparkfun_edge_cortex-  
m4/bin/micro_speech.bin -O binary
```

如要確認二進位檔是否已順利建立，請執行下列指令：

```
test -f \  
tensorflow/lite/micro/tools/make/gen/sparkfun_edge_cortex-m4/bin/micro_speech.bin &&  
\  
echo "Binary was successfully created" || echo "Binary is missing"
```

控制台應該會顯示 `Binary was successfully created`！如果看到 `Binary is missing`，表示建構程序發生問題，需要進行偵錯。

準備二進位檔

二進位檔必須以加密金鑰簽署，才能部署到裝置。現在我們要執行一些指令，簽署二進位檔，以便下載至 SparkFun Edge。

輸入下列指令，設定一些可供開發使用的虛擬加密金鑰：

```
cp tensorflow/lite/micro/tools/make/downloads/AmbiqSuite-  
Rel2.2.0/tools/apollo3_scripts/keys_info0.py  
tensorflow/lite/micro/tools/make/downloads/AmbiqSuite-  
Rel2.2.0/tools/apollo3_scripts/keys_info.py
```

現在，請執行下列指令來建立已簽署的二進位檔：

```
python3 tensorflow/lite/micro/tools/make/downloads/AmbiqSuite-  
Rel2.2.0/tools/apollo3_scripts/create_cust_image_blob.py \  
--bin tensorflow/lite/micro/tools/make/gen/sparkfun_edge_cortex-  
m4/bin/micro_speech.bin \  
--load-address 0xC000 \  
--magic-num 0xCB \  
-o main_nonsecure_ota \  
--version 0x0
```

這樣會建立 `main_nonsecure_ota.bin` 檔案。現在我們要執行另一個指令，建立最終版本的檔案，可用於透過開機載入程式指令碼刷寫裝置，我們會在下一個步驟使用該指令碼：

```
python3 tensorflow/lite/micro/tools/make/downloads/AmbiqSuite-  
Rel2.2.0/tools/apollo3_scripts/create_cust_wireupdate_blob.py \  
--load-address 0x20000 \  
--bin main_nonsecure_ota.bin \  
-i 6 \  
-o main_nonsecure_wire \  
--options 0x1
```

現在，您執行指令的目錄中應該會有一個名為 `main_nonsecure_wire.bin` 的檔案。這是我們要刷入裝置的檔案。

步驟 5 · 約 5 分鐘

5. 前置準備刷入二進位檔

什麼是刷機？

SparkFun Edge 會將目前執行的程式儲存在 512 KB 的快閃記憶體中。如要讓開發板執行新程式，我們必須將程式傳送至開發板，開發板會將程式儲存在快閃記憶體中，覆寫先前儲存的任何程式。

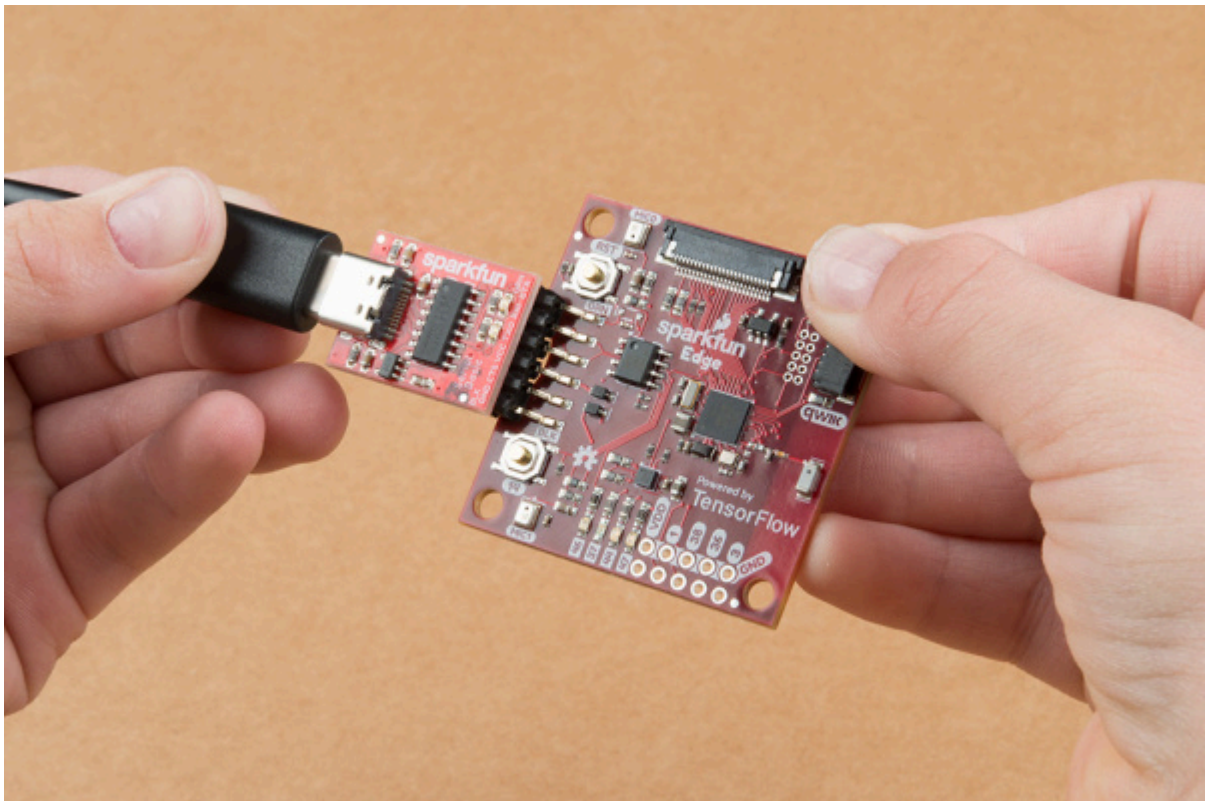
這個程序稱為「刷機」，我們會用來將程式傳送至開發板。

將程式設計器連接至開發板

我們會使用 SparkFun USB-C Serial Basic 序列埠程式設計器，將新程式下載到開發板。這部裝置可讓電腦透過 USB 與微控制器通訊。

如要將裝置連接至開發板，請按照下列步驟操作：

1. 找出 SparkFun Edge 側邊的六針接頭。
2. 將 SparkFun USB-C Serial Basic 插入這些接腳，確保每個裝置上標示為「BLK」和「GRN」的接腳正確對齊。



將程式設計器連接到電腦

我們會透過 USB 將開發板連接至電腦。如要為開發板編程，我們需要知道電腦為裝置指定的名稱。最好的做法是列出連接前後的所有電腦裝置，然後找出新裝置。

透過 **USB** 連接裝置前，請先執行下列指令：

```
If you are using Linux: ls /dev/tty*  
If you are using MacOS: ls /dev/cu*
```

這會輸出附加裝置清單，類似於下列內容：

```
/dev/cu.Bluetooth-Incoming-Port  
/dev/cu.MALS  
/dev/cu.SOC
```

現在，將程式設計器連接至電腦的 **USB 連接埠**。再次輸入下列指令：

```
If you are using Linux: ls /dev/tty*  
If you are using MacOS: ls /dev/cu*
```

輸出內容中應會多出一個項目，如下例所示。**新項目名稱可能不同**。這個新項目是裝置名稱。

```
/dev/cu.Bluetooth-Incoming-Port  
/dev/cu.MALS  
/dev/cu.SOC  
/dev/cu.wchusbserial-1450
```

注意：部分使用者回報清單中出現兩部裝置。如果看到兩個裝置，請選取開頭為「**wch**」的裝置，例如「**/dev/wchusbserial-1450**」。

首先，我們會建立環境變數來識別裝置名稱：

```
export DEVICENAME=put your device name here
```

注意：在本範例中，指令為：`export DEVICENAME=/dev/cu.wchusbserial-1450`

接下來，我們要建立環境變數來指定鮑率，也就是資料傳送至裝置的速度：

```
export BAUD_RATE=921600
```

注意：如果您是在 **TensorFlow** 開發人員高峰會上取得 SparkFun Edge 開發板，代表您使用的是早期版本的硬體，通訊時會使用不同的鮑率。請改用下列指令：

```
export BAUD_RATE=115200
```

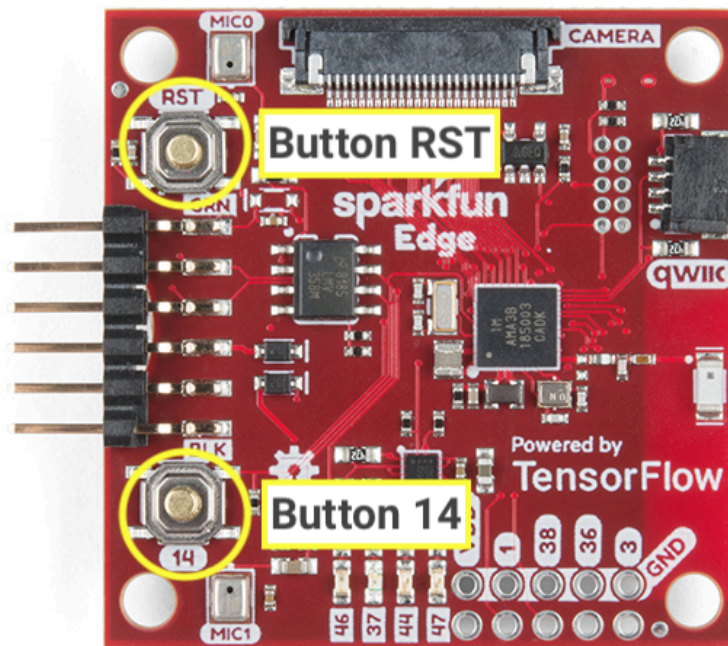
步驟 6 · 約 10 分鐘

6. 刷入二進位檔

執行指令碼來刷寫開發板

如要刷新開發板，我們必須將其設為特殊的「系統啟動載入程式」狀態，準備接收新的二進位檔。接著，我們會執行指令碼，將二進位檔傳送至開發板。

讓我們熟悉白板上的下列按鈕：



請按照下列步驟重設並刷寫開發板：

1. 確認開發板已連線至程式設計師，且整個設定透過 USB 連接至電腦。
2. 開始按住電路板上標示 14 的按鈕。請繼續按住按鈕，直到步驟 6。
3. 按住標示 14 的按鈕，然後點選標示 RST 的按鈕，將開發板重設為開機載入程式狀態。
4. 按住標示 14 的按鈕，將下列指令貼到終端機，然後按下 Enter 鍵執行指令 (為方便起見，您可以在開始按住按鈕前，將此指令貼到終端機，但請等到這個步驟再按下 Enter 鍵)

```
python3 tensorflow/lite/micro/tools/make/downloads/AmbiqSuite-  
Rel2.2.0/tools/apollo3_scripts/uart_wired_update.py -b ${BAUD_RATE} ${DEVICENAME} -r  
1 -f main_nonsecure_wire.bin -i 6
```

5. 繼續按住標示 14 的按鈕，螢幕上應會顯示類似下列的內容：

```
Connecting with Corvette over serial port /dev/cu.usbserial-1440...
Sending Hello.
Received response for Hello
Received Status
length = 0x58
version = 0x3
Max Storage = 0x4ffa0
Status = 0x2
State = 0x7
AMInfo =
0x1
0xff2da3ff
0x55fff
0x1
0x49f40003
0xffffffff
[...lots more 0xffffffff...]
Sending OTA Descriptor = 0xfe000
Sending Update Command.
number of updates needed = 1
Sending block of size 0x158b0 from 0x0 to 0x158b0
Sending Data Packet of length 8180
Sending Data Packet of length 8180
[...lots more Sending Data Packet of length 8180...]
```

6. 看到 `Sending Data Packet of length 8180` 後，**停止** 按住電路板上標示 14 的按鈕 (繼續按住也沒關係)。程式會繼續在終端機上列印行。最終會顯示類似下列內容：

```
[...lots more Sending Data Packet of length 8180...]
Sending Data Packet of length 8180
Sending Data Packet of length 6440
Sending Reset Command.
Done.
```

如果看到 `Done`，表示**成功**閃爍。如果程式輸出內容以錯誤結尾，請檢查是否已列印 `Sending Reset Command`。如果是，即使發生錯誤，韌體可能也已順利更新。

注意：下列內容僅適用於擁有 Linux 電腦，但無法刷新裝置的使用者。否則，請繼續下一節。

在 Linux 電腦上，您可能會遇到 `NoResponse Error`。這是因為 [ch34x 序列驅動程式](#) 已與現有的序列驅動程式一併安裝，您可以按照下列步驟解決這個問題：

步驟 1：重新安裝正確版本的 ch34x 程式庫。安裝時，請務必將裝置從電腦拔除。

```
git clone https://github.com/juliagoda/CH341SER.git
cd CH341SER/
make
sudo insmod ch34x.ko
sudo rmmod ch341
```

步驟 2：插入開發板 USB 並執行下列指令：

```
dmesg | grep "ch34x"
```

畫面上應會顯示類似下方的訊息：

```
[ 1299.444724] ch34x_attach+0x1af/0x280 [ch34x]
[ 1299.445386] usb 2-13.1: ch34x converter now attached to ttyUSB0
```

如果使用的驅動程式不是「ch34x」(例如 ch341)，請執行以下指令停用其他驅動程式：

```
rmmod <non-ch34x driver name>
```

拔除裝置並重新插入，確認使用的驅動程式為「ch34x」。

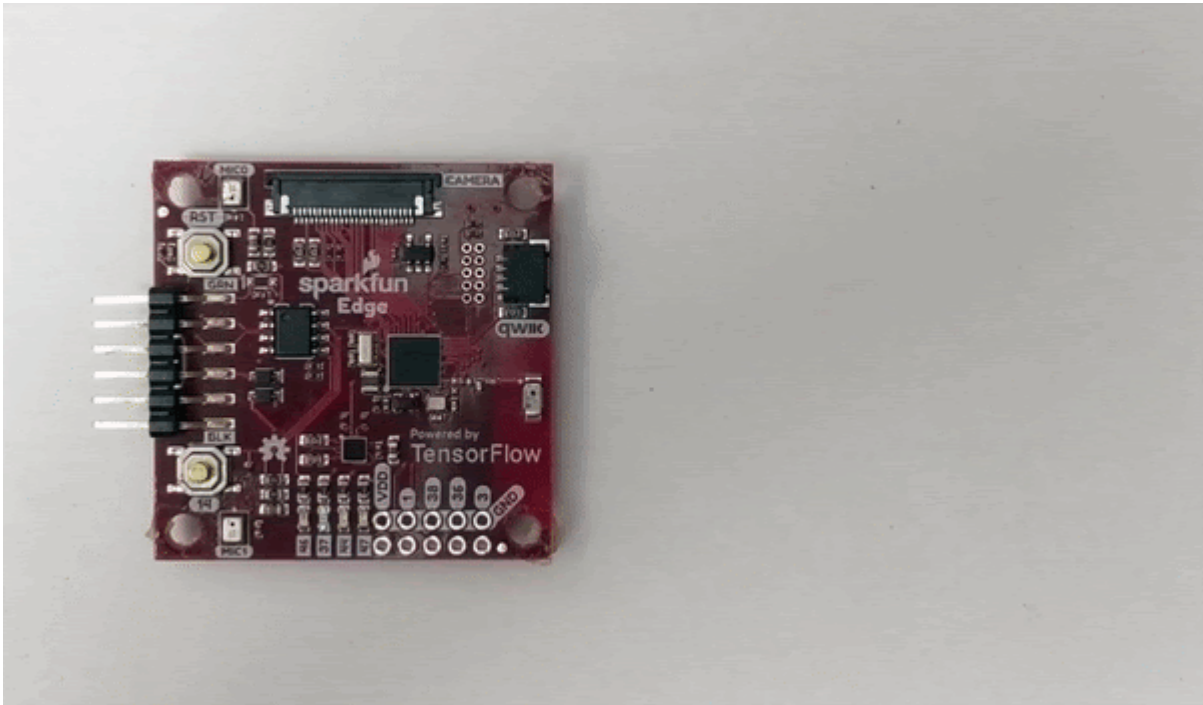
步驟 7 · 約 5 分鐘

7. 示範

試用這項計畫

順利刷寫開發板後，按下標示為

RST 重新啟動開發板並啟動程式。如果藍色 LED 燈開始閃爍，表示閃爍成功。如果沒有，請向下捲動至下方的「如果無法解決問題？」一節。



開發板上的機器學習模型經過訓練，可辨識「是」和「否」這兩個字，並偵測語音。並透過亮起彩色 LED 燈來傳達結果。下表說明各個 LED 燈色的意義：

偵測結果	LED 燈顏色
「是」	黃色
「否」	紅色
不明語音	綠色
偵測不到語音	LED 燈未亮起

歡迎體驗

將板子舉到嘴巴前，然後說幾次「是」。黃色 LED 燈會閃爍。如果說「是」後沒有任何反應，請嘗試下列做法：

- 將板子放在距離嘴巴約 10 吋的位置
- 避免背景噪音過多

- 快速連續說出「是」幾次 (嘗試說出「是是是」)

如果無法解決問題，該怎麼辦？

以下列出一些可能的問題及除錯方式：

問題：閃爍後，**LED** 燈完全沒有亮起。

解決方法：嘗試按下 `RST` 按鈕，或中斷開發板與程式設計師的連線，然後再重新連線。如果上述方法都無效，請再次刷寫開發板。

問題：藍色 **LED** 燈亮起，但亮度很低。

解決方法：更換電池，因為電量不足。或者，您也可以使用程式設計器和傳輸線，透過電腦為開發板供電。

步驟 8 · 約 0 分鐘

8. 讀取偵錯輸出內容 (選用)

如果遇到問題，需要詳細偵錯程式碼，請參閱本節。如要瞭解程式碼執行時微控制器發生的情況，可以透過開發板的序列連線列印偵錯資訊。使用電腦連線至開發板，並顯示開發板傳送的資料。

開啟序列連線

根據預設，SparkFun Edge 程式碼範例會記錄所有語音指令及其信賴度。如要查看開發板的輸出內容，請執行下列指令：

```
screen ${DEVICENAME} 115200
```

您一開始可能會看到類似下列範例的輸出內容 (只有在主機板連線後重設時才會顯示，否則您可能會開始看到偵錯資訊)：

```
Apollo3 Burst Mode is Available
```

```
Apollo3 operating in Burst Mode (96MHz)
```

請說「是」或「否」，發出一些指令。您應該會看到每個指令的電路板顯示偵錯資訊：

```
Heard yes (202) @65536ms
```

在上述記錄中，`yes` 是指指令。數字 `202` 代表系統對指令辨識結果的信賴程度 (最低為 200)。最後，`65536ms` 是指微控制器上次重設後經過的時間長度。

如要停止查看偵錯輸出內容，請依序按下 `Ctrl+A`、`K` 和 `Y` 鍵。

寫入偵錯記錄

您可以在剛才使用的 `command_responder.cc` 檔案中，查看記錄這項資訊的程式碼：

```
tensorflow/lite/micro/examples/micro_speech/sparkfun_edge/command_responder.cc
```

如要記錄資料，可以呼叫 `error_reporter->Report()` 方法。它支援字串插補的標準 `printf` 權杖，可用於在記錄中加入重要資訊：

```
error_reporter->Report("Heard %s (%d) @%dms", found_command, score, current_time);
```

在下一節中，您將自行變更程式碼，屆時這個方法應該會派上用場。

9. 延長代碼效期 (選用)

現在您已瞭解如何建構及刷新 SparkFun Edge，可以開始使用程式碼並部署至裝置，查看結果。

讀取程式碼

建議您從以下檔案開始閱讀程式碼：`command_responder.cc`。

```
tensorflow/lite/micro/examples/micro_speech/sparkfun_edge/command_responder.cc
```

您可以在 GitHub 上查看檔案，請參閱[這裡](#)。

偵測到語音指令時，系統會呼叫這個檔案中的 `RespondToCommand` 方法。現有程式碼會根據系統是否聽到「是」、「否」或不明指令，開啟不同的 LED 燈。以下程式碼片段說明如何運作：

```
if (found_command[0] == 'y') {
    am_hal_gpio_output_set(AM_BSP_GPIO_LED_YELLOW);
}
if (found_command[0] == 'n') {
    am_hal_gpio_output_set(AM_BSP_GPIO_LED_RED);
}
if (found_command[0] == 'u') {
    am_hal_gpio_output_set(AM_BSP_GPIO_LED_GREEN);
}
```

`found_command` 引數包含偵測到的指令名稱。這組 `if` 陳述式會檢查第一個字元，判斷要點亮哪個 LED。

系統會使用多個引數呼叫 `RespondToCommand` 方法：

```
void RespondToCommand(tflite::ErrorReporter* error_reporter,
    int32_t current_time, const char* found_command,
    uint8_t score, bool is_new_command) {
```

- `error_reporter` 用於記錄偵錯資訊 (稍後會詳細說明)。
- `current_time` 代表系統偵測到指令的時間。
- `found_command` 會顯示偵測到的指令。
- `score` 代表我們偵測到指令的信心程度。
- `is_new_command` 會告訴我們這是第一次聽到指令。

`score` 是介於 0 到 255 的整數，代表偵測到指令的機率。程式碼範例只會將分數大於 200 的指令視為有效。根據測試結果，大多數有效指令都落在 200 到 210 的範圍內。

修改程式碼

SparkFun Edge 開發板有四個 LED。目前我們是以藍色 LED 燈閃爍表示正在進行辨識。您可以在

`command_responder.cc` 檔案中看到這點：

```
static int count = 0;

// Toggle the blue LED every time an inference is performed.
++count;
if (count & 1) {
    am_hal_gpio_output_set(AM_BSP_GPIO_LED_BLUE);
} else {
    am_hal_gpio_output_clear(AM_BSP_GPIO_LED_BLUE);
}
```

我們有四個 LED 燈，因此請修改程式，將這些 LED 燈做為特定指令 `score` 的視覺指標。分數越低，亮起的 LED 燈就越少；分數越高，亮起的 LED 燈就越多。

為確保我們能知道程式正在執行，我們會讓紅色 LED 燈持續閃爍，而不是藍色。相鄰的藍色、綠色和黃色 LED 燈會顯示最近一次 `score` 的強度。為求簡單，我們只會在說出「yes」這個字時點亮 LED。如果偵測到其他字詞，LED 燈會熄滅。

如要進行這項變更，請將 `command_responder.cc` 檔案中的所有程式碼替換為下列程式碼片段：

```

#include "tensorflow/lite/micro/examples/micro_speech/command_responder.h"

#include "am_bsp.h"

// This implementation will light up the LEDs on the board in response to different commands.
void RespondToCommand(tflite::ErrorReporter* error_reporter,
                     int32_t current_time, const char* found_command,
                     uint8_t score, bool is_new_command) {
    static bool is_initialized = false;
    if (!is_initialized) {
        // Setup LEDs as outputs
        am_hal_gpio_pinconfig(AM_BSP_GPIO_LED_RED, g_AM_HAL_GPIO_OUTPUT_12);
        am_hal_gpio_pinconfig(AM_BSP_GPIO_LED_BLUE, g_AM_HAL_GPIO_OUTPUT_12);
        am_hal_gpio_pinconfig(AM_BSP_GPIO_LED_GREEN, g_AM_HAL_GPIO_OUTPUT_12);
        am_hal_gpio_pinconfig(AM_BSP_GPIO_LED_YELLOW, g_AM_HAL_GPIO_OUTPUT_12);
        // Ensure all pins are cleared
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_RED);
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_BLUE);
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_GREEN);
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_YELLOW);
        is_initialized = true;
    }
    static int count = 0;

    // Toggle the red LED every time an inference is performed.
    ++count;
    if (count & 1) {
        am_hal_gpio_output_set(AM_BSP_GPIO_LED_RED);
    } else {
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_RED);
    }

    if (is_new_command) {
        // Clear the last three LEDs
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_BLUE);
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_GREEN);
        am_hal_gpio_output_clear(AM_BSP_GPIO_LED_YELLOW);
        error_reporter->Report("Heard %s (%d) @%dms", found_command, score,
                               current_time);

        // Only indicate a 'yes'
        if (found_command[0] == 'y') {
            // Always light the blue LED
            am_hal_gpio_output_set(AM_BSP_GPIO_LED_BLUE);
            // Light the other LEDs depending on score
            if (score >= 205) {
                am_hal_gpio_output_set(AM_BSP_GPIO_LED_GREEN);
            }
            if (score >= 210) {
                am_hal_gpio_output_set(AM_BSP_GPIO_LED_YELLOW);
            }
        }
    }
}

```

```
}  
  
}
```

如果偵測到新指令，`is_new_command` 會是 `true`。我們會清除藍色、綠色和黃色 LED，然後根據 `found_command` 和 `score` 的值再次點亮。

重建並刷新

程式碼變更完成後，請執行「建構及準備二進位檔」中的所有步驟，測試變更是否正常運作。

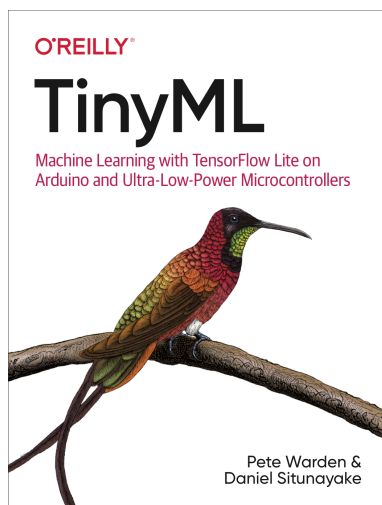
10. 後續步驟

恭喜，您已成功在微控制器上建構第一個語音偵測器！

希望您喜歡這篇簡短的 TensorFlow Lite for Microcontrollers 開發入門教學。在微控制器上進行深度學習是令人興奮的新概念，我們鼓勵您勇於嘗試！

參考文件

- 現在您已熟悉基本程式，可以[訓練自己的模型](#)，瞭解不同的指令。*注意：訓練作業需要幾小時才能完成！*
- 進一步瞭解 TensorFlow Lite for Microcontrollers ([網站](#)、[GitHub](#))。
- 嘗試其他[範例](#)，並在支援的 SparkFun Edge 上執行。
- 請參閱 O'Reilly 出版的《[TinyML: Machine Learning with TensorFlow on Arduino and Ultra-Low Power Micro-Controllers](#)》一書，瞭解如何在微型裝置上進行機器學習，並逐步完成幾個有趣的專案。本程式碼研究室是以書中第 7 章和第 8 章為基礎。



感謝你，祝你建構愉快！
